Research Square

Preprints are preliminary reports that have not undergone peer review.
They should not be considered conclusive, used to inform clinical practice,
or referenced by the media as validated information.

Systematic Evaluation of AI-Generated Python Code: A Comparative Study across Progressive Programming Tasks

Yang Qianyi

y02214625@stu.ahu.edu.cn

Anhui University

Research Article

Keywords:

Posted Date: September 23rd, 2024

DOI: https://doi.org/10.21203/rs.3.rs-4955982/v1

License: This work is licensed under a Creative Commons Attribution 4.0 International License.
Read Full License

Additional Declarations: No competing interests reported.

Abstract

Background: AI-based code assistants are on the rise in software development as powerful technologies offering streamlining of code generation and better-quality code. However, their effectiveness is very variable, and understanding their pros and cons becomes very important in regard to using them optimally.

Introduction: This study evaluates the capabilities of four prominent AI-based code assistants: GitHub Copilot, Microsoft Copilot, Tabnine, and ChatGPT. This study addresses whether the quality of produced code is functional, efficient, and maintainable and whether it has areas for improvement.

Methodology: The AI-generated code was compared in terms of correctness, McCabe complexity (cyclomatic complexity), efficiency, and code size. The correctness percentage was the portion of code without errors, McCabe complexity was used for measuring structural complexity, execution performance represented efficiency, and the size of the code was just by lines of code. All AI tools were benchmarked against a standard set of 100 prompts to ensure like-for-like assessment.

Results: GitHub Copilot had the highest correctness at 42%, and ChatGPT generated the most complex code—which was measured with a McCabe complexity score of 2.92. Efficiencywise, ChatGPT also topped with the highest number of codes that meet the "good" criteria. On average, Tabnine produced the shortest code, whereas GitHub Copilot and ChatGPT were among the most verbose. The analysis revealed that although AI-based assistants can generate high-quality code, they usually produce code far different from solutions written by developers themselves, and it is difficult for them to cope with dependencies between classes.

Conclusion: AI-based code assistants have substantial potential for improvingcode generation and software development efficiency. However, challenges remain, particularly in handling complex dependencies and producing ready-to-use code. The study suggests that leveraging the strengths of different assistants and focusing on enhancing their ability to manage complex coding scenarios could lead to significant advancements. Ongoing research and development are essential to address these limitations and fully harness the potential of AI-based code assistants in software development.

Background

Systematic evaluation and benchmarking of deep reinforcement learning algorithms for continuous control tasks have been research topics with many activities. With a wide benchmark setup over a comprehensive range of tasks, researchers can understand not only the strong aspects of these algorithms but also their limitations in guiding possible advancements in the future. The proposed approach not only enables the identification of new strategies and optimizations that can be added to reinforcement learning methods for possible performance gain but also makes it possible to carry out more pragmatic implementations, leading to greater robustness and efficiency in both academic and industrial environments [1]. As the landscape of deep reinforcement learning rapidly changes, it is

increasingly important to adhere to solid experimental techniques and good standards for reporting so that the results can be reproduced and form a basis for meaningful comparisons between algorithms and task performance [2].

Recently, generative AI has proven to be a very powerful tool for complementing and enhancing deep reinforcement learning, particularly for the generation of very complex models and simulations. This involves generative models that create complex, high-fidelity environments that match real-world conditions as closely as possible; thus, more challenging and realistic tasks can be used to train reinforcement learning models [3]. This has been leveraged for overall improvements in both training efficiency and generalization capabilities by the synergy of generative AI with reinforcement learning, allowing better performance on a broader range of tasks. Moreover, generative AI offers new opportunities for data augmentation and scenario generation—one of the main necessities for robustness testing of reinforcement learning models in new and previously unknown settings [4].

In this sense, the adoption of a systematic approach to experimentation and evaluation will not only answer critical questions about reproducibility but also create an environment in which diverse efforts can draw on one another, thus driving the field forward [5]. To that end, we propose a list of recommendations that rely on significance metrics and common standards of reporting together with making the integrity and interpretability of experimental findings stand at the heart of this work in deep reinforcement learning. While generative AI is evolving, its amalgamation with reinforcement learning techniques seems to lead to innovative adaptations in shaping the future of AI research and applications.

Introduction

One of the most disruptive forces to have developed over the past few years by means of generative AI is code development. Generative AI is a relatively new area of artificial intelligence whose models are designed to produce new content on the basis of learned patterns from provided data [6]. Many fruitful applications have gone through and boiled down to automating and improving the coding process.

This has been very beneficial for generating code snippets, complete functions, and even whole applications with less time and effort in software development. With the integration of this generative AI into the workflow, programmers are expected to work much more effectively, innovatively, and at a greatly increased scale [7]. The adoption of generative AI in code development represents not only a trend but also a paradigm shift in uniting the power of machine learning with the precision of programming. This synergy has opened new avenues for creating robust, efficient, and maintainable code, paving the way for advanced software solutions that may adapt to complex and evolving requirements.

Generative AI Models for Code Development

GitHub Copilot: GitHub, in collaboration with OpenAI, has developed GitHub Copilot, an AI-powered code completion tool designed to integrate seamlessly with popular code editors such as Visual Studio Code [8]. This tool provides contextually relevant code suggestions, generates boilerplate code, and assists in writing complex functions. GitHub Copilot is built on OpenAI's Codex model, which has been trained on a vast dataset of public code repositories. Its primary objective is to aid developers by offering code snippets and completing code on the basis of the current context.

Microsoft Copilot: The Microsoft Copilot application, created by GitHub and fueled by the OpenAI Codex, is an AI tool that guides developers by suggesting line codes, completing function arguments, and even creating entire code blocks. The integration of this application with standard but very popular code editors, including Visual Studio Code, will make it fully available to any wide range of developers without any hustle. Copilots enhance productivity by providing context-aware code suggestions, which allows developers to engage in higher-level logic as AI becomes more routine and repetitive [9].

Tabnine: Tabnine is an AI code completion tool that uses deep learning algorithms to predict and suggest code snippets while typing. It is applicable to many programming languages and integrates many IDEs [10]. Code writing is highly efficient because of its intelligent code completion, which, on the basis of the context of the code, saves many manual types and maintains standards for writing code.

Blackbox AI: Blackbox AI made by Meta is a generative AI tool that enables developers to write code with real-time suggestions on the basis of the context of the code they are currently writing. It is designed to integrate with numerous IDEs and support many programming languages [11]. Blackbox AI helps reduce the coding time, as it gives you the relevant code-snippets and autocompletions, mostly for repetitive works.

Llama 3.0: Llama 3.0 is a large, state-of-the-art language model developed by Meta (formerly Facebook) that generates human-like text and code. It has not been trained directly on generating code, but the powerful language model is a tool valuable to developers aiming at automating some part of the workflow in coding [12]. Llama 3.0 can help create the code by producing documentation and even comments and portions of code on the basis of elaborate descriptions given by the programmer.

Replit AI: Replit AI is an in-IDE, AI-powered coding assistant that extends the Replit platform to a fully fledged IDE that supports collaborative code writing. Through code suggestions, autocompletion, and even debugging help, which it provides, the user truly can [13]. Repliting AI enhances real-time suggestions and corrections for collaborative coding, specifically those involving teams working on the same set of projects.

ChatGPT: ChatGPT is an OpenAI conversational AI model developed in a GPT style and put into use toward natural language processing, especially in coding-oriented parts of text generation for users to assist them in their queries with code generation [14]. ChatGPT supports developers

in obtaining answers to any of their coding queries, generates solutions with code snippets on the basis of user input, and explains complex programming concepts.

Gemini: Gemini is an advanced AI model developed by Google that is used to assist many things, including code generation. It uses much research from Google on machine learning to provide accurate and contextual code suggestions [15]. Gemini is used for generating code, automating redundant tasks, and enabling intelligent code completions that speed up developer work.

These are state-of-the-art models in generative AI of code development, each with unique features and abilities that cater to a different aspect of the software development process. These tools clearly promise a lead role in creating the future of coding and software engineering.

Methodology

We systematically reviewed the performance of a variety of AI-powered code assistants over 100 carefully prepared Python prompts in this study [16]. These prompts were set to include everything from the simplest functions to complex algorithms. The main aim of this evaluation was to understand what these AI models can do in generating proper and effective code.

Code Documentation

All the code generated by AI-based assistants was properly documented inside a Jupyter Notebook, making it highly structured and organized. This approach made it much easier to refer back and forth between related code files for comparison across different models. Each prompt, along with its corresponding AI-generated code, was captured, along with additional metadata such as the time of code generation and code complexity metrics.

Research Questions

To evaluate the quality and practicality of the generated code, we focused on four critical research questions:

Correctness and Implement ability

We checked the work produced by the AI-based assistants to see if it was valid and if it could be turned into working code by a developer. This involved running the code to check for errors, testing its functionality against expected outcomes, and determining whether the code could be integrated into a larger project with minimal modification.

McCabe Complexity and Code Generation Time

We consider the McCabe complexity, or cyclomatic complexity, of the generated code. This metric sheds light on the maintainability and readability of the code. Additionally, we monitored how long each AI took to generate the code, which highlighted issues related to efficiency in the code generation process.

Efficiency

We benchmarked the efficiency of the generated code on the basis of runtime performance by comparing it with standard implementations and assessing how well the AI-generated code optimized computational resources.

Size of the generated code

The size of the generated code was measured in terms of lines of code (LOC) and byte size. This metric helps to understand the verbosity or conciseness of the code produced by each AI, which can impact both readability and performance.

The research questions stated above aimed to provide a comprehensive understanding of the efficiency of AI-based code assistants in coding scenarios. In this section, we document and analyze the results across these dimensions to provide valuable insights for both academic research and practical applications in the domain of software development.

Results

GitHub Copilot emerged as the leader in assessing the correctness and implementability of code generated by various AI-based assistants, with a correctness rate of 42%. This means that most of the time, the code produced by the GitHub Copilot was correct and could be integrated into larger projects with minimal modification. In contrast, Table 9 had the lowest correctness rate at 19%, indicating that its generated code often required significant adjustments or failed to meet the expected outcomes. Microsoft Copilot and ChatGPT presented correctness rates of 35% and 33%, respectively. Blackbox AI, Llama 3.0, Replit AI, and Gemini also performed reasonably well but fell short of the top performers. These results reveal varying levels of reliability among AI code generators, with GitHub Copilot showing the most promise in generating functional and accurate code. A visual representation of the results is shown in Table 1 and Fig. 1.

Table 1. Artificial Intelligence Code Performance Metrics

Sr. no.	Name of code generating AI	Average Lines of Code (Mean)	Correctness of the Code (Percentage)	McCabe Complexity Score	Code Generation Time (seconds)	Efficiency or time complexity of the code (Number of Code)		
						Good	Fair	Bad
1	Github Copilot	22.23	42	2.15	4	62	29	9
2	Microsoft Copilot	18.84	35	2.8	3	68	23	9
3	Tabnine	16.26	19	2.89	3	73	17	10
4	Blackbox AI	18.43	23	2.23	4	67	27	6
5	Llama 3.0	18.09	25	2.25	3	66	26	8
6	Replit AI	18.89	20	2.29	4	70	21	9
7	Chat Gpt	23.47	33	2.92	5	76	15	9
8	Gemini	19.01	30	2.78	3	72	18	10

The size of the code generated by various AI-based assistants was assessed by measuring both the average number of lines of code (LOC) and overall byte size, providing insights into the verbosity and conciseness of the code. GitHub Copilot produced an average of 22.23 lines of code, making it one of the more verbose options. In contrast, Tabnine generated the shortest average code at 16.26 lines, indicating a more concise output. ChatGPT's generated code was the longest, averaging 23.47 lines, suggesting more elaborate or detailed implementations. Microsoft Copilot, Blackbox AI, Llama 3.0, Replit AI, and Gemini generated code with averages ranging from 18.09 to 19.01 lines, reflecting a moderate level of verbosity. These results highlight differences in code size among AI assistants, with some favoring brevity and others tending to produce more extensive code, potentially impacting readability and performance. A visual representation of the results is shown in Table 1 and Fig. 1.

The analysis of McCabe complexity and code generation time provides valuable insights into the structural complexity and efficiency of code generated by various AI-based assistants. GitHub Copilot achieved the lowest McCabe complexity score at 2.15, indicating that the generated code was the least complex and potentially the easiest to maintain and read. In terms of the code generation time, the GitHub Copilot also performed efficiently, with an average time of 4 seconds. This suggests that the GitHub Copilot not only produces less complex code but also does so relatively quickly. Conversely, ChatGPT presented the highest McCabe complexity score at 2.92 and the longest generation time of 5 s, suggesting that its code was more complex and took more time to generate. Other assistants, such as Microsoft Copilot, Tabnine, and Gemini, showed a balance between complexity and generation time, with complexity scores ranging from 2.23 to 2.89 and generation times ranging from 3 to 4 seconds. These findings highlight the trade-offs between code complexity and generation efficiency among different AI-based code assistants. A visual representation of the results is shown in Table 1 and Fig. 2.

The efficiency of the code generated by AI-based assistants was evaluated on the basis of its execution performance, with a focus on how well the code optimized computational resources. The GitHub Copilot showed strong performance, with 62 codes falling into the "good" criterion, indicating that its generated code was highly efficient. In comparison, ChatGPT led with the highest number of codes in the "good" category, totaling 76, reflecting superior execution performance. Moreover, 73 codes met the "good" criteria, whereas Microsoft Copilot and Replit AI had 68 and 70 codes, respectively. Conversely, the number of codes categorized under the "Fair" and "Bad" criteria varied, with ChatGPT having the fewest codes falling into the "Fair" category (15) and the same number in the "Bad" category (9) as other assistants did. This assessment highlights the varying levels of efficiency among AI-based assistants, with some showing notable strengths in generating high-performance code, whereas others performed less optimally. A visual representation of the results is shown in Table 1 and Fig. 3.

The results clearly indicate that AI-based code assistants can be valuable tools for developers but also highlight areas needing improvement. While these assistants generally generate code that is correct and efficient, they also produce a considerable amount of invalid or incorrect code, particularly for methods involving interclass dependencies. Additionally, the code generated by AI-based assistants often differs significantly from the code written by developers, necessitating substantial revisions. These findings

underscore the need for further development and refinement of AI-based code assistants to increase their accuracy, efficiency, maintainability, and alignment with developer-written code.

Discussion

The findings of this research have several implications for both research and practice. Collaboration among multiple AI-based code assistants appears to be a promising direction for future development. By leveraging the strengths of different assistants, developers can enhance code generation and mitigate the limitations of individual tools. The results show that while GitHub Copilot and ChatGPT excel in correctness and efficiency, other assistants, such as Tabnine and Microsoft Copilot, also contribute valuable insights, highlighting the potential benefits of combining their outputs.

Second, the study revealed that some AI-based code assistants can produce code that surpasses the quality of what developers might write. This finding is crucial for improving code quality and efficiency in software development, as certain AI-generated code examples have proven to be highly effective and performant.

Third, the challenges related to interclass dependencies underscore the need for further improvement in AI-based code assistants. The study highlighted that AI tools often struggle with methods involving complex interclass interactions, suggesting that there is significant potential for enhancing these tools to handle such dependencies more effectively. Future research and development should focus on empowering AI capabilities to address these complex coding scenarios.

AI-based code assistants hold significant potential for improving code generation and quality. However, ongoing research is essential to address the existing challenges and limitations. This study investigated the capabilities of four AI-based code assistants—GitHub Copilot, Microsoft Copilot, Tabnine, and ChatGPT—and compared them across functional correctness, complexity, efficiency, size, and similarity to original developer-written code. The findings reveal that while these assistants demonstrate complementary capabilities, with GitHub Copilot generating the highest rate of correct methods, the generated code often serves as a good starting point rather than a ready-to-use solution. Addressing limitations, particularly in managing interclass dependencies, remains a key area for future improvement.

Conclusion

In conclusion, these findings highlight the significant potential of AI-powered code assistants to improve coding and software development practices. While these tools provide invaluable assistance in generating correct and efficient code, they also have limitations, particularly in managing complex interclass dependencies, and the code they produce often requires substantial revision. The comparative analysis of GitHub Copilot, Microsoft Copilot, Tabnine, and ChatGPT shows that while each assistant has strengths—such as GitHub Copilot's superior correctness rate and ChatGPT's efficiency—the generated code often deviates from what developers actually want, necessitating further refinement.

Future research should focus on improving AI-based code assistants to better handle these challenges, especially in managing intricate dependencies and reducing the need for extensive modifications. By continuing to refine their accuracy, efficiency, and maintainability, there is potential to significantly enhance the quality and effectiveness of AI-generated code. Overall, while AI-based code assistants are promising tools for developers, ongoing research and development are crucial to fully realize their potential and integrate them effectively into the software development process.

Declarations

Author Contribution

Author Yang Qian yi did everything all

Data Availability

https://github.com/YangQianYi-AHU/AI-Python-Code-Evaluation

References

1. Singh, V., Chen, S. S., Singhania, M., Nanavati, B., Kar, A. K., & Gupta, A. (2022). How are reinforcement learning and deep learning algorithms used for big data based decision making in financial industries–A review and research agenda. *International Journal of Information Management Data Insights*, *2*(2), 100094. https://doi.org/10.1016/j.jjimei.2022.100094
2. Han, D., Mulyana, B., Stankovic, V., & Cheng, S. (2023). A Survey on Deep Reinforcement Learning Algorithms for Robotic Manipulation. *Sensors*, *23*(7), 3762. https://doi.org/10.3390/s23073762
3. Liu, Y., Yang, Z., Yu, Z., Liu, Z., Liu, D., Lin, H., Li, M., Ma, S., Avdeev, M., & Shi, S. (2023). Generative artificial intelligence and its applications in materials science: Current situation and future perspectives. *Journal of Materiomics*, *9*(4), 798–816. https://doi.org/10.1016/j.jmat.2023.05.001
4. Linkon, N. a. A., Shaima, N. M., Sarker, N. M. S. U., Badruddowza, N., Nabi, N. N., Rana, N. M. N. U., Ghosh, N. S. K., Esa, N. H., & Chowdhury, N. F. R. (2024). Advancements and Applications of Generative Artificial Intelligence and Large Language Models on Business Management: A Comprehensive Review. *Journal of Computer Science and Technology Studies*, *6*(1), 225–232. https://doi.org/10.32996/jcsts.2024.6.1.26
5. Mariani, M. M., Machado, I., Magrelli, V., & Dwivedi, Y. K. (2023). Artificial intelligence in innovation research: A systematic review, conceptual framework, and future research directions. *Technovation*, *122*, 102623. https://doi.org/10.1016/j.technovation.2022.102623
6. Lv, Z. (2023). Generative artificial intelligence in the metaverse era. *Cognitive Robotics*, *3*, 208–217. https://doi.org/10.1016/j.cogr.2023.06.001

7. *The Role of Generative AI in Software Development Productivity: A Pilot Case Study*. (n.d.). https://arxiv.org/html/2406.00560v1
8. Kalliamvakou, E. (2024, May 21). *Research: quantifying GitHub Copilot's impact on developer productivity and happiness - The GitHub Blog*. The GitHub Blog. https://github.blog/news-insights/research/research-quantifying-github-copilots-impact-on-developer-productivity-and-happiness/
9. Cambon, A., & Hecht, B. (2023). *Early LLM-based Tools for Enterprise Information Workers Likely Provide Meaningful Boosts to Productivity*. https://www.microsoft.com/en-us/research/uploads/prod/2023/12/AI-and-Productivity-Report-First-Edition.pdf
10. *Blog | Tabnine: The AI coding assistant that you control*. (2024, July 25). Tabnine. https://www.tabnine.com/blog/
11. *Chat Blackbox: AI Code Generation, Code Chat, Code Search*. (n.d.). https://www.blackbox.ai/blog
12. Blog | Introducing Meta Llama 3: The most capable openly available LLM to date https://ai.meta.com/blog/meta-llama-3/
13. Replit. (2023, October 9). *Replit — Announcing Replit AI for All*. Replit Blog. https://blog.replit.com/ai4all
14. Research Blog | Hello GPT-°We're announcing GPT-4o, our new flagship model that can reason across audio, vision, and text in real time. https://openai.com/index/hello-gpt-4o/ (13 May 2024)
15. Research Blog | Introducing Gemini: our largest and most capable AI model https://blog.google/technology/ai/google-gemini-ai/(06 December 2023)
16. AI-Python-Code-Evaluation , https://github.com/YangQianYi-AHU/AI-Python-Code-Evaluation https://github.com/YangQianYi-AHU/AI-Python-Code-Evaluation (22 August 2024)

Figures

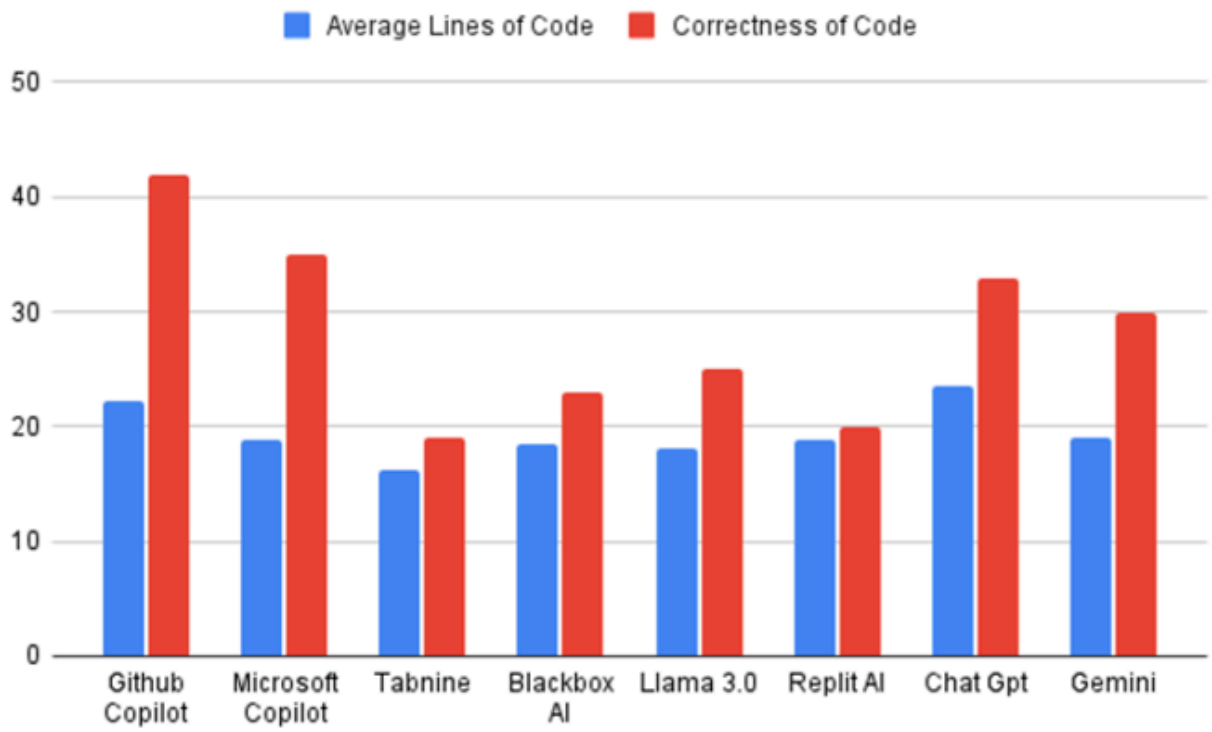

Figure 1. Average lines of code in Mean and Correctness of code in percentage in each individual model

Figure 1

See image above for figure legend.

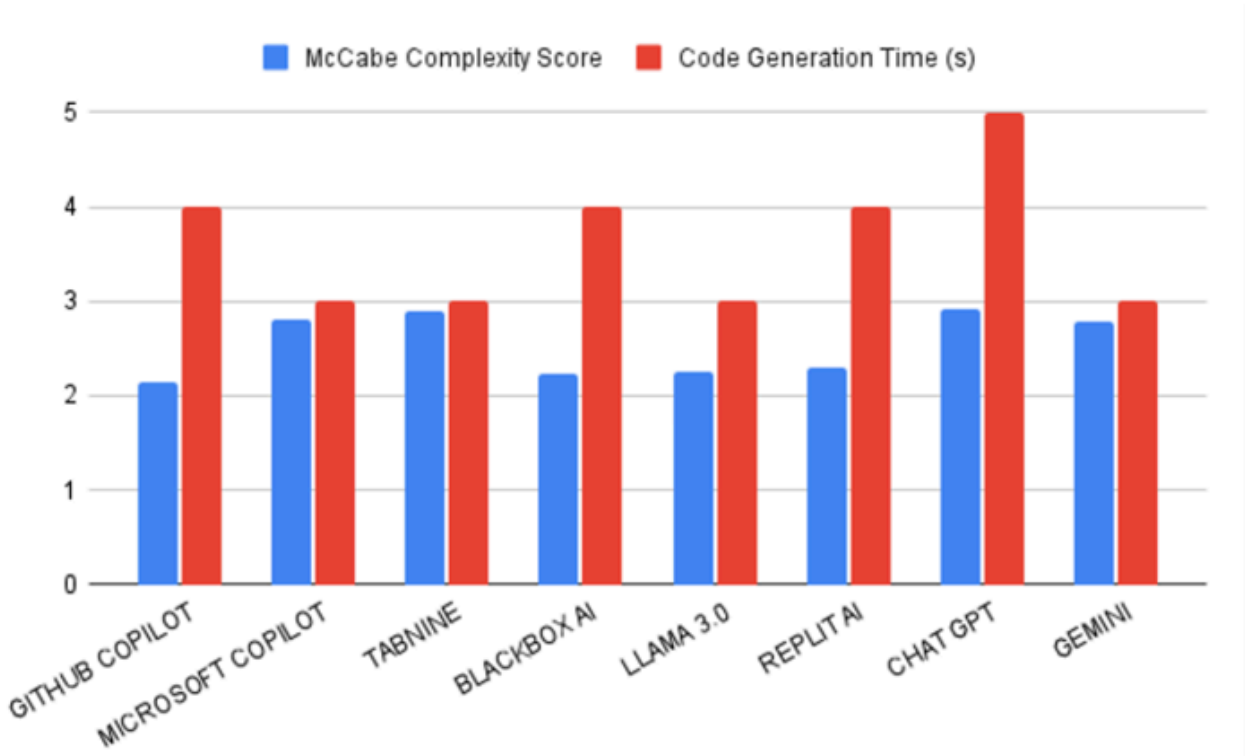

Figure 2 . McCabe Complexity Score and Code Generation Time (s) in each individual model.

Figure 2

See image above for figure legend.

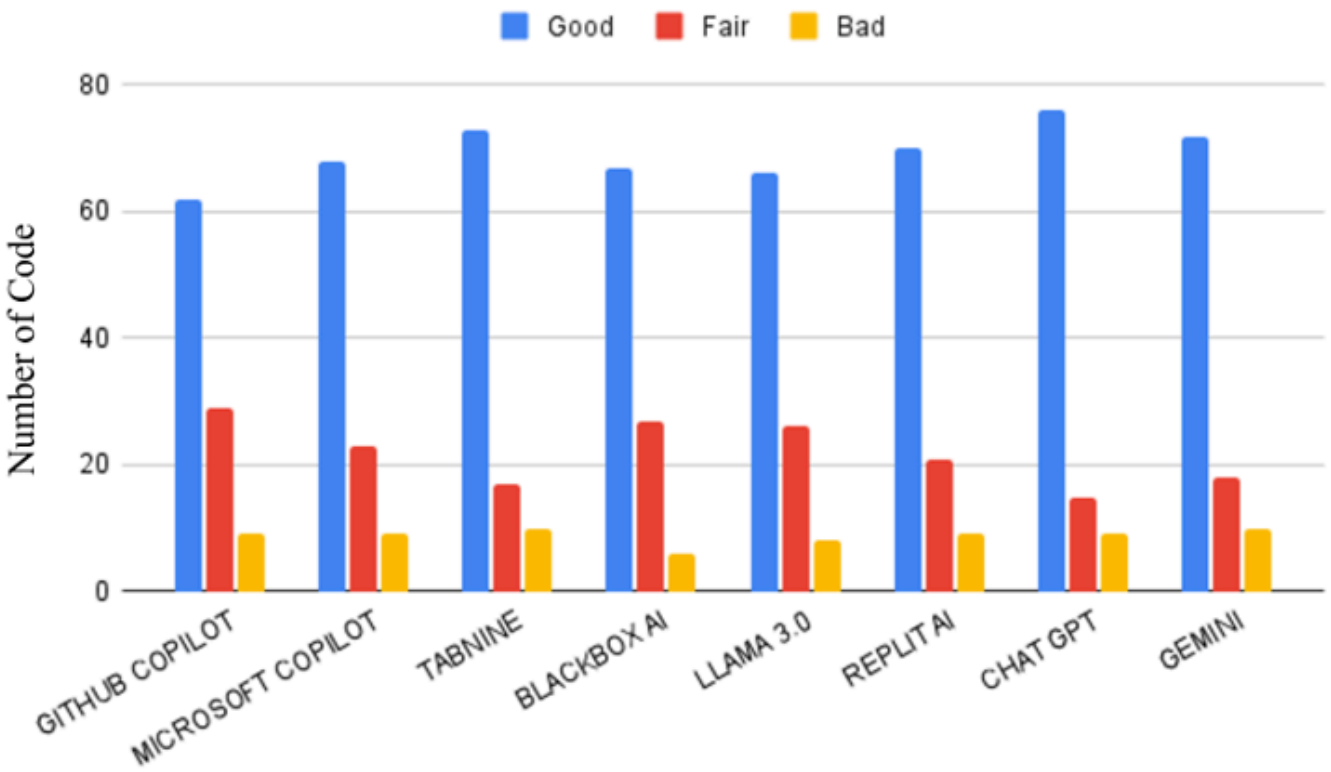

Figure 3 . Efficiency or time complexity of the code in each individual model.

Figure 3

See image above for figure legend.